AUTOMATION FOR JAVA DEVELOPERS · SELENIUM

Selenium with Java: From Zero to Real Tests

You already know Java. This guide teaches only what is new: Selenium words, objects and methods, and how to use them to test a real web page. Every code example is Java and was run successfully.

IMPORTANT

Red boxes and rows: learn these first. They are used in almost every test and asked in almost every interview.

GOOD TO KNOW

Blue boxes and rows: useful, but you can learn them later or only when needed.

---

1. What is Selenium and how does it work?

Selenium is a Java library (a set of classes in a jar file) that controls a real browser. Your Java code says "open this page", "type here", "click that button", and the browser does it. Then your code checks the result.

Your Java test → **Selenium library** → **Browser driver (chromedriver)** → **Chrome browser**

Selenium cannot talk to Chrome directly. A small program called the **browser driver** sits in the middle and translates the commands. In Selenium 4, a helper called **Selenium Manager** downloads the correct driver for you automatically.

Words you will hear

WORD	SIMPLE MEANING
WebDriver	The main Java interface. One WebDriver object = one browser you can control.
Driver / ChromeDriver	`WebDriver` is the interface. `ChromeDriver` is the class that opens Chrome. `FirefoxDriver` and `EdgeDriver` do the same for other browsers.
WebElement	One thing on the page: a button, text box, link, label. Your code holds it in a `WebElement` variable.

Locator (By)	The way to tell Selenium which element you mean: by id, name, CSS, XPath and so on.
Session	From `new ChromeDriver()` to `driver.quit()`. One browser window lifetime.
DOM	The page as a tree of elements. Locators search this tree.
Headless	Running the browser with no visible window. Faster, used on servers.
Selenium Grid	Runs tests on many machines and browsers at once. Good to know only.
Selenium IDE	A browser plugin that records clicks. Not used in real projects.

---

2. Setup in Java (Maven)

IMPORTANT **Dependencies**

Create a Maven project, then add these dependencies to `pom.xml`. Maven downloads the jars for you.

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.27.0</version>
</dependency>
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.10.2</version>
</dependency>
```

What it does: `selenium-java` is Selenium itself. `testng` is the test framework that runs your tests and gives you `Assert`. Use a recent Selenium version if you see a newer one on Maven Central. A "CDP version" warning in the console is harmless.

Also install Google Chrome. Nothing else is needed, Selenium Manager gets the driver.

Your first test (complete)

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class FirstTest {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();                  // 1. open Chrome
        driver.get("https://example.com");                       // 2. go to a page
        String title = driver.getTitle();                        // 3. read the page
        System.out.println(title);
        driver.findElement(By.tagName("a")).click();             // 4. find a link a
        driver.quit();                                           // 5. close the bro
    }
}
```

What it does: Five steps that every Selenium test has: open the browser, go to a page, find elements and act on them, check something, close the browser.

IMPORTANT **Why WebDriver on the left side?**

`WebDriver driver = new ChromeDriver();` The variable type is the interface `WebDriver`, the object is `ChromeDriver`. This is Java polymorphism. To run on Firefox you only change the right side to `new FirefoxDriver()`. Everything else stays the same.

IMPORTANT **Never forget quit()**

Always call `driver.quit()`. If you forget it, browser windows stay open and eat memory. In a test class, put it in `@AfterMethod` so it runs even when the test fails.

---

3. All the main classes in one picture

CLASS / INTERFACE	USE
`WebDriver`	Controls the browser: open pages, find elements, switch windows.
`WebElement`	One element: click it, type in it, read its text.

`By`	Describes how to find an element (id, CSS, XPath...).
`Select`	Helper for `<select>` dropdowns.
`Actions`	Mouse and keyboard actions: hover, double click, drag and drop.
`Alert`	Browser popups (alert, confirm, prompt).
`WebDriverWait` + `ExpectedConditions`	Wait until something is true.
`JavascriptExecutor`	Run JavaScript inside the page.
`Keys`	Special keys: ENTER, TAB, BACK_SPACE...
`ChromeOptions`	Settings for Chrome: headless, window size, notifications off.

---

4. WebDriver methods (the browser)

These are called on the `driver` object. Red rows you will use daily.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`get(String url)`	Open a web page and wait until it loads.	`driver.get("https://shop.com");`
IMPORTANT	`getTitle()`	Returns the page title.	`String t = driver.getTitle();`
IMPORTANT	`getCurrentUrl()`	Returns the current address.	`driver.getCurrentUrl();`
IMPORTANT	`findElement(By)`	Finds the first matching element. Throws `NoSuchElementException` if none.	`driver.findElement(By.id("email"));`
IMPORTANT	`findElements(By)`	Finds all matching elements as a `List`. Empty list if none (no error).	`List<WebElement> rows = driver.findElements(By.tagName("tr"));`

IMPORTANT	`quit()`	Closes every window and ends the session.	`driver.quit();`
IMPORTANT	`close()`	Closes only the current window or tab.	`driver.close();`
IMPORTANT	`navigate().back() / forward() / refresh() / to(url)`	Browser back, forward, reload, go to a URL.	`driver.navigate().back();`
IMPORTANT	`manage().window().maximize()`	Makes the window full size.	`driver.manage().window().maximize();`
IMPORTANT	`manage().timeouts().implicitlyWait(Duration)`	Global wait for finding elements (see waits section).	`driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));`
IMPORTANT	`switchTo().frame(...)`	Go inside an iframe. Use name, id, index or WebElement.	`driver.switchTo().frame("payFrame");`
IMPORTANT	`switchTo().defaultContent()`	Come back from a frame to the main page.	`driver.switchTo().defaultContent();`
IMPORTANT	`switchTo().window(String handle)`	Move to another window or tab.	`driver.switchTo().window(handle);`
IMPORTANT	`switchTo().alert()`	Take control of a browser popup.	`Alert a = driver.switchTo().alert();`
IMPORTANT	`getWindowHandle()` / `getWindowHandles()`	Id of the current window / ids of all open windows (a `Set<String>`).	`Set<String> all = driver.getWindowHandles();`
GOOD TO KNOW	`getPageSource()`	The full HTML of the page as a String.	`driver.getPageSource();`
GOOD TO KNOW	`switchTo().newWindow(WindowType.TAB)`	Open a new tab and move to it.	`driver.switchTo().newWindow(WindowType.TAB);`

GOOD TO KNOW	`switchTo().parentFrame()`	Go up one frame level.	`driver.switchTo().parentFrame();`
GOOD TO KNOW	`manage().window().minimize()` / `fullscreen()` / `setSize()`	Other window controls.	`driver.manage().window().setSize(new Dimension(1280, 800));`
GOOD TO KNOW	`manage().timeouts().pageLoadTimeout(Duration)`	How long to wait for a page to load before failing.	
GOOD TO KNOW	`manage().getCookies()` / `addCookie()` / `deleteAllCookies()`	Read, add and clear cookies. `deleteAllCookies()` gives a fresh start.	`driver.manage().deleteAllCookies();`

IMPORTANT **close() vs quit()**

`close()` closes one window. `quit()` closes everything. Interviewers ask this difference very often.

IMPORTANT **findElement vs findElements**

`findElement` gives one element and fails if none is found. `findElements` gives a list and never fails. Use `findElements(...).size() == 0` to check that something is NOT on the page.

---

5. Locators: how to find an element

A web page is HTML. Each element has attributes like `id`, `name`, `class`. To click a button, you first find it with a locator from the `By` class.

```
<input id="email" name="email" type="email" class="form-control" placeholder="you@ex
<button type="submit" class="btn">Login</button>
```

What it does: Example HTML we will find elements in.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`By.id("email")`	Finds by the id attribute. Fastest and best when the id is unique and stable.	`driver.findElement(By.id("email"))`
IMPORTANT	`By.cssSelector("input#email")`	Finds with a CSS selector. Fast and flexible.	`By.cssSelector("button[type='submit']")`
IMPORTANT	`By.xpath("//button[text()='Login']")`	Finds with XPath. Most powerful: can use text and parent / child.	`By.xpath("//input[@id='email']")`
IMPORTANT	`By.name("email")`	Finds by the name attribute.	`By.name("email")`
IMPORTANT	`By.linkText("Forgot password?")`	Finds a link by its full visible text.	`By.linkText("Sign up")`
GOOD TO KNOW	`By.partialLinkText("Forgot")`	Link text that contains the given part.	
GOOD TO KNOW	`By.className("btn")`	Finds by one class name. Many elements often share a class.	`By.className("btn")`
GOOD TO KNOW	`By.tagName("a")`	Finds by tag. Used to get lists like all links or all rows.	`driver.findElements(By.tagName("a"))`
GOOD TO KNOW	`RelativeLocator.with(By.tagName("input")).below(By.id("email"))`	Selenium 4: find by position (above, below, toLeftOf, toRightOf, near).	

CSS and XPath cheat sheet

WANT TO FIND	CSS SELECTOR	XPATH
By id	`#email`	`//*[@id='email']`
By class	`.btn`	`//*[@class='btn']`
Tag with attribute	`input[type='email']`	`//input[@type='email']`

By exact text	not possible	`//button[text()='Login']`
By part of text	not possible	`//button[contains(text(),'Log')]`
Part of an attribute	`div[class*='err']`	`//div[contains(@class,'err')]`
Starts with	`input[id^='user']`	`//input[starts-with(@id,'user')]`
Child inside parent	`#product-3 button`	`//div[@id='product-3']//button`
Second item of many	`li:nth-child(2)`	`(//li)[2]`
Next to a label	not possible	`//label[text()='Email']/following::input[1]`
Two conditions	`input[type='radio'][value='UPI']`	`//input[@type='radio' and @value='UPI']`

IMPORTANT **Locator rules**

- Order of choice: **id** first, then **CSS**, then **XPath**.
- XPath that starts with `//` is **relative** and safe. XPath like `/html/body/div[2]/div[1]/button` is **absolute** and breaks when the page changes a little.
- Get a locator: right click the element in Chrome, click Inspect, read its attributes. Test your locator in the Console: `$x("//button[text()='Login']")` for XPath and `$$("#email")` for CSS.
- If an id changes on every load (like `btn_8374`), use `starts-with()` or `contains()`.

---

6. WebElement methods (the things on the page)

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`click()`	Clicks the element.	`driver.findElement(By.id("loginBtn")).click();`

IMPORTANT	`sendKeys(CharSequence...)`	Types text. Also sends special keys. On a file input, sends the file path.	`box.sendKeys("hello");`
IMPORTANT	`clear()`	Removes the text of an input.	`box.clear();`
IMPORTANT	`getText()`	Returns the visible text of the element.	`String msg = el.getText();`
IMPORTANT	`getAttribute("name")`	Returns an attribute or property such as `value`, `href`, `class`, `type`. To read what a user typed, use `getAttribute("value")`.	`box.getAttribute("value");`
IMPORTANT	`isDisplayed()`	True if the element is visible on the page.	`el.isDisplayed();`
IMPORTANT	`isEnabled()`	True if the element is not disabled. Use for buttons.	`btn.isEnabled();`
IMPORTANT	`isSelected()`	True if a checkbox, radio button or option is selected.	`box.isSelected();`
GOOD TO KNOW	`getDomAttribute("name")`	Selenium 4: the attribute exactly as written in the HTML.	`el.getDomAttribute("data-state");`
GOOD TO KNOW	`getDomProperty("name")`	Selenium 4: the live property value of the element.	`el.getDomProperty("value");`
GOOD TO KNOW	`getTagName()`	The tag, such as `input`.	
GOOD TO KNOW	`getCssValue("color")`	A style value. Used for colour or font checks.	`el.getCssValue("color");`
GOOD TO KNOW	`getSize()` / `getLocation()` / `getRect()`	Width, height and position on the page.	
GOOD TO KNOW	`submit()`	Submits the form that contains the element.	
GOOD TO KNOW	`findElement(By)` / `findElements(By)`	Search only inside this element (a child search).	`card.findElement(By.tagName("button"));`

GOOD TO KNOW	`getScreenshotAs(OutputType.FILE)`	Screenshot of just this element.

IMPORTANT **Common mix-ups**

- `getText()` gives text you can SEE. A hidden element returns an empty string.
- `getAttribute("value")` gives what is INSIDE a text box. `getText()` on a text box returns empty. This confuses everyone once.
- `isDisplayed()` throws `NoSuchElementException` if the element is not in the HTML at all. Use `findElements(...).size()` for that case.

---

7. Special elements

Dropdown: the Select class

Works only for the HTML `<select>` tag. Wrap the element in `Select`.

```
Select country = new Select(driver.findElement(By.id("country")));
country.selectByVisibleText("India");      // choose by the text the user sees
country.selectByValue("USA");              // choose by the value attribute
country.selectByIndex(0);                  // choose by position, starts at 0
String chosen = country.getFirstSelectedOption().getText();
```

What it does: Selects an option in three ways and reads the chosen one.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`selectByVisibleText(String)`	Choose by the visible text.	
IMPORTANT	`selectByValue(String)`	Choose by the value attribute.	
IMPORTANT	`getOptions()`	List of all options as `List<WebElement>`.	`select.getOptions().size();`
IMPORTANT	`getFirstSelectedOption()`	The selected option (single dropdown).	

GOOD TO KNOW	`selectByIndex(int)`	Choose by position.
GOOD TO KNOW	`isMultiple()`	True for a multi-select list.
GOOD TO KNOW	`getAllSelectedOptions()`	All selected options of a multi-select.
GOOD TO KNOW	`deselectAll()` / `deselectByValue()` / `deselectByVisibleText()`	Only for multi-select lists.

Popups: the Alert interface

```
Alert alert = wait.until(ExpectedConditions.alertIsPresent());   // wait for the po
String text = alert.getText();     // read the message
alert.accept();                    // press OK
// alert.dismiss();                // press Cancel
// alert.sendKeys("abc");          // type into a prompt box
```

What it does: Waits for a popup, reads it, and presses OK.

IMPORTANT **Alert rule**

While a popup is open, Selenium cannot do anything else on the page. You will get `UnhandledAlertException`. Always handle the popup first.

Mouse and keyboard: the Actions class

```
Actions actions = new Actions(driver);
actions.moveToElement(menu).perform();                // hover
actions.doubleClick(button).perform();                // double click
actions.contextClick(item).perform();                 // right click
actions.dragAndDrop(source, target).perform();        // drag and drop
actions.sendKeys(Keys.ESCAPE).perform();              // press a key
```

What it does: Every Actions chain must end with perform(), or nothing happens.

Special keys

```
box.sendKeys("selenium", Keys.ENTER);                    // type and press Enter
box.sendKeys(Keys.BACK_SPACE);                           // delete one character
box.sendKeys(Keys.chord(Keys.CONTROL, "a"));             // Ctrl + A (use Keys.COMMA
```

What it does: Keys has ENTER, TAB, ESCAPE, BACK_SPACE, ARROW_UP, ARROW_DOWN and more.

JavascriptExecutor

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].scrollIntoView(true);", element);      // scroll to
js.executeScript("arguments[0].click();", element);                   // click with
js.executeScript("arguments[0].value = arguments[1];", dateBox, "2000-05-20");   //
Object title = js.executeScript("return document.title;");
```

What it does: Runs JavaScript inside the page. `arguments[0]` is the first value you pass after the script text.

Use it when normal Selenium cannot do the job: date pickers, hidden elements, scrolling, or a click that is blocked by another element. Try normal Selenium first.

Frames, windows and file upload

```
driver.switchTo().frame("payFrame");           // go inside an iframe
driver.switchTo().defaultContent();             // come back to the main page

String main = driver.getWindowHandle();          // remember the main window id
for (String h : driver.getWindowHandles()) {
    if (!h.equals(main)) driver.switchTo().window(h);   // move to the new tab
}

driver.findElement(By.id("resume")).sendKeys("/Users/me/resume.pdf");   // file upl
```

What it does: For an `<input type="file">` you do NOT click. You send the full path of the file as text.

Screenshot

```
File shot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
Files.copy(shot.toPath(), Path.of("failure.png"));
```

What it does: Saves a picture of the page. Normally done when a test fails.

---

8. Waits: the most important topic

IMPORTANT **Why we need waits**

A page needs time to load. If your code looks for a button before it appears, you get `NoSuchElementException`. A **wait** tells Selenium to give the page a little time. Without waits, tests fail randomly.

WAIT	HOW IT WORKS	ADVICE
Implicit	One global rule: for every `findElement`, keep trying up to N seconds.	Simple, but do not mix with explicit waits.
Explicit (`WebDriverWait`)	Wait for ONE condition on ONE element (visible, clickable...).	Best choice. Use this.
Fluent	Explicit wait where you also set how often to check and which errors to ignore.	For special cases.
Thread.sleep	Stops for a fixed time.	Avoid. It always waits the full time.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
```

What it does: Implicit wait: for the whole test, find calls retry for up to 5 seconds.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement msg = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("ms
wait.until(ExpectedConditions.elementToBeClickable(By.id("payBtn"))).click();
```

What it does: Waits up to 10 seconds, but continues the moment the condition is true. If it never becomes true, a `TimeoutException` is thrown.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`visibilityOfElementLocated(By)`	Element is in the page and visible. Returns the element.	
IMPORTANT	`elementToBeClickable(By)`	Element is visible and enabled, ready to click.	
IMPORTANT	`presenceOfElementLocated(By)`	Element exists in the HTML (it may still be hidden).	
IMPORTANT	`alertIsPresent()`	A popup is open.	
GOOD TO KNOW	`invisibilityOfElementLocated(By)`	Element has gone (for example a loading spinner).	
GOOD TO KNOW	`textToBePresentInElementLocated(By, text)`	Element contains the given text.	
GOOD TO KNOW	`titleContains(text)` / `urlContains(text)`	Page title or URL has the text.	
GOOD TO KNOW	`frameToBeAvailableAndSwitchToIt(...)`	Frame is ready and Selenium switches into it.	
GOOD TO KNOW	`numberOfElementsToBe(By, n)`	Exactly n elements found.	
GOOD TO KNOW	`stalenessOf(element)`	The old element is gone from the page.	

```
Wait<WebDriver> fluent = new FluentWait<>(driver)
        .withTimeout(Duration.ofSeconds(20))
        .pollingEvery(Duration.ofMillis(500))
        .ignoring(NoSuchElementException.class);
WebElement el = fluent.until(d -> d.findElement(By.id("late")));
```

What it does: Fluent wait: check every 500 ms for up to 20 seconds, and ignore "not found" errors while waiting.

9. Common exceptions

	EXCEPTION	WHY IT HAPPENS	FIX
IMPORTANT	`NoSuchElementException`	Element not found. Wrong locator, page not loaded, or inside a frame.	Fix the locator, add a wait, switch to the frame.
IMPORTANT	`TimeoutException`	An explicit wait never became true.	Check condition and locator.
IMPORTANT	`StaleElementReferenceException`	The page refreshed and your saved element is old.	Find the element again.
IMPORTANT	`ElementClickInterceptedException`	Another element (banner, popup) covers the button.	Close the cover, scroll, or wait.
GOOD TO KNOW	`ElementNotInteractableException`	Element exists but is hidden or disabled.	Wait until visible or enabled.
GOOD TO KNOW	`NoAlertPresentException`	You switched to an alert but none is open.	Wait for `alertIsPresent()`.
GOOD TO KNOW	`UnhandledAlertException`	A popup is open and blocks the next action.	Accept or dismiss it first.
GOOD TO KNOW	`SessionNotCreatedException`	Browser and driver versions do not match.	Update Selenium or Chrome.

10. TestNG: running and checking tests

Selenium only controls the browser. **TestNG** runs your methods as tests and tells you pass or fail. It works like JUnit with extra features.

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`@Test`	Marks a method as a test.	
IMPORTANT	`@BeforeMethod / @AfterMethod`	Run before and after EACH test. Use for opening and closing the browser.	

IMPORTANT	`@DataProvider`	Gives many rows of data to one test method (data-driven testing).	`@Test(dataProvider = "logins")`
GOOD TO KNOW	`@BeforeClass / @AfterClass`	Run once for the whole class.	
GOOD TO KNOW	`@BeforeSuite / @AfterSuite`	Run once for the whole run.	
GOOD TO KNOW	`@Test(priority = 1, enabled = false, groups = "smoke")`	Order, skipping and grouping of tests.	
GOOD TO KNOW	`testng.xml`	A file to choose which tests to run, run in parallel, or pass parameters.	

	METHOD	WHAT IT DOES	EXAMPLE
IMPORTANT	`Assert.assertEquals(actual, expected)`	Fails if the two values differ. Note the order: actual first, expected second.	`Assert.assertEquals(driver.getTitle(), "Home");`
IMPORTANT	`Assert.assertTrue(condition) / assertFalse`	Fails if the condition is not true / not false.	`Assert.assertTrue(btn.isEnabled());`
IMPORTANT	`Assert.assertEquals(actual, expected, "message")`	The message shows when the test fails. Always add one for clear reports.	
GOOD TO KNOW	`Assert.assertNotNull / assertNull / fail()`	Other checks.	
GOOD TO KNOW	`SoftAssert`	Collects ALL failures and reports them together when you call `assertAll()`. A normal Assert stops at the first failure.	`soft.assertAll();`

The base class used in the three cases below

```java
package selenium;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

import java.time.Duration;

public class BaseTest {

    protected WebDriver driver;
    protected WebDriverWait wait;

    @BeforeMethod
    public void setUp() {
        ChromeOptions options = new ChromeOptions();
        if (Boolean.getBoolean("headless")) {
            options.addArguments("--headless=new");      // run without showin
        }
        options.addArguments("--window-size=1280,900");
        driver = new ChromeDriver(options);               // opens Chrome
        wait = new WebDriverWait(driver, Duration.ofSeconds(5));
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        if (driver ≠ null) {
            driver.quit();                                // closes the browse
        }
    }
}
```

LINES	WHAT THIS PART DOES
1-11	Package and imports: classes borrowed from Selenium, TestNG and Java.
12-13	The parent class of all test classes. Other classes say `extends BaseTest` and get the browser setup for free (inheritance).
14-16	`driver` controls the browser. `wait` is a ready explicit wait of 5 seconds. They are `protected` so child classes can use them.
17-18	TestNG runs this before every test method.

19-23	Settings for Chrome. With `-Dheadless=true` Chrome runs without a window. The window size is fixed so the layout is the same on every machine.
24	Starts Chrome. Selenium Manager downloads the right chromedriver if needed.
25-27	Creates the wait: up to 5 seconds for any condition.
28-34	Runs after every test, even if the test failed (`alwaysRun = true`). `quit()` closes the browser and ends the session.

11. Page Object Model (design pattern)

GOOD TO KNOW **POM**

Without POM, every test has locators inside it. If a locator changes, you edit many tests. In POM each page gets one class holding its locators and actions. Tests only call these actions. The three cases below keep things simple, so the locators and helper methods sit inside each test class. A real project would move them into page classes like this:

```
public class LoginPage {
    private final WebDriver driver;
    private final By email = By.id("email");
    private final By password = By.id("pass");
    private final By loginBtn = By.id("loginBtn");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public void loginAs(String user, String pass) {
        driver.findElement(email).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(loginBtn).click();
    }
}
// in a test:  new LoginPage(driver).loginAs("qa@test.com", "Test@1234");
```

What it does: Locators are private and hidden (encapsulation). If the id changes, you fix one line.

12. Three real automations

Click a case below to open it. In each case you will see: the practice page, where every element is, the manual test cases, the full Java code with line numbers, and the code explained line by line. **Read the manual test cases first, then the code.** That is how real automation is done: manual thinking first, code second.

IMPORTANT **Before you start**

- Open each practice page in Chrome and test it by hand first. The page links are inside each case below.
- `PAGE_URL` in each test already points to the live practice page on GitHub Pages, so you only need internet. To use your own copy, open the file from `docs/demo-pages/` and put its `file:///` path in `PAGE_URL`.
- Run with Maven: `mvn test -Dtest=PasswordTest`. Remove `-Dheadless=true` to watch the browser work.
- All three cases were run and passed (66 checks) on the pages as given.

▼ Case 1: Password validation (a full validation spec)

Boundary values, equivalence partitions, negative and state testing on one password form.

STEP 1: THE PRACTICE PAGE

Open the page, click around like a manual tester first, then look at the locators below.

Open demo-password.html

The test already points to this live page (`PAGE_URL`), so it runs from any computer with internet. The same file is in the repository at `docs/demo-pages/demo-password.html`.

IMPORTANT **What you will learn from this case**

- Data-driven tests (one method runs many times with different data)
- Boundary value analysis and equivalence partitioning in real code
- Checking one rule at a time
- Reading attributes and the enabled / disabled state of a button

STEP 2: WHERE EACH ELEMENT IS (LOCATORS)

ELEMENT	HTML ON THE PAGE	BY.ID	CSS SELECTOR	XPATH
Username box	`<input id="username" type="text">`	`username`	`#username`	`//input[@id='username']`
Password box	`<input id="password" type="password">`	`password`	`input[type='password']`	`//input[@id='password']`
Confirm password box	`<input id="confirmPassword" type="password">`	`confirmPassword`	`#confirmPassword`	`//label[text()='Confirm password']/following::input[1]`
Show / Hide button	`<button id="togglePwd">Show</button>`	`togglePwd`	`#togglePwd`	`//button[text()='Show']`
One rule line	`<li id="rule-length" data-state="pending">`	`rule-length`	`li[data-state='pass']`	`//li[@id='rule-length']`
Strength text	`<span id="strength">Not entered</span>`	`strength`	`#strength`	`//span[@id='strength']`
Error message	`<div id="pwdError" class="error">`	`pwdError`	`#pwdError`	`//div[@id='pwdError']`
Create password button	`<button type="submit" id="submitBtn" disabled>`	`submitBtn`	`button[type='submit']`	`//button[@type='submit']`
Success message	`<div id="successMsg" hidden>`	`successMsg`	`#successMsg`	`//div[contains(text(),'created successfully')]`

STEP 3: MANUAL TEST CASES FIRST

Automation follows manual testing. We first decide what to test, and only then write code.

ID	TECHNIQUE USED	INPUT	EXPECTED RESULT
TC01	State check	Nothing typed	Button disabled, strength "Not entered", rules pending
TC02-05	Positive + boundary	Abcd@1234, Abc@1234 (8 chars), Abcdefgh@1234567 (16 chars), Str0ng@Passw0rd	All rules pass, Medium or Strong, button enabled, success message
TC06-07	Boundary value	7 characters, 17 characters	Only the length rule fails
TC08-11	Equivalence partition	No uppercase, no lowercase, no digit, no special character	Only that one rule fails
TC12	Error guessing	A space inside the password	No-space rule fails
TC13	Business rule	Username ubed, password MyUBED@123	No-username rule fails
TC14-16	Error guessing (hostile input)	SQL text, script text, only spaces	Rejected, button disabled, no popup
TC17-19	Negative: confirm password	Different, different only in case, empty	Error message, button disabled
TC20-21	State transition	Valid then invalid then valid; username changed later	Button enabled or disabled to match the data
TC22	UI behaviour	Click Show / Hide	Field type switches between password and text

STEP 4: THE CODE

File: `selenium/PasswordTest.java`

```java
package selenium;

import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

public class PasswordTest extends BaseTest {

    // Live practice page hosted on GitHub Pages (works from any computer
    private static final String PAGE_URL = "https://ubedzz49.github.io/qa-

    private static final String[] ALL_RULES =
            {"length", "upper", "lower", "number", "special", "nospace", "

    // ---------- Locators: all in one place ----------
    private final By username = By.id("username");
    private final By password = By.id("password");
    private final By confirm = By.id("confirmPassword");
    private final By submitBtn = By.id("submitBtn");
    private final By toggleBtn = By.id("togglePwd");
    private final By strength = By.id("strength");
    private final By pwdError = By.id("pwdError");
    private final By confirmError = By.id("confirmError");
    private final By successMsg = By.id("successMsg");

    // ---------- Helper methods ----------
    private void open() {
        driver.get(PAGE_URL);
    }

    private void type(By locator, String text) {
        WebElement field = driver.findElement(locator);
        field.clear();
        if (!text.isEmpty()) {
            field.sendKeys(text);
        }
    }

    private void enter(String pwd, String confirmPwd, String user) {
        type(username, user);
        type(password, pwd);
        type(confirm, confirmPwd);
    }

    private void enter(String pwd) {
        enter(pwd, pwd, "");
```

```
50      }
51
52      private String ruleState(String rule) {
53          return driver.findElement(By.id("rule-" + rule)).getDomAttribute("
54      }
55
56      private boolean buttonEnabled() {
57          return driver.findElement(submitBtn).isEnabled();
58      }
59
60      // ---------- 1. Initial state ----------
61      @Test
62      public void tc01_initialState() {
63          open();
64          Assert.assertFalse(buttonEnabled(), "Button must be disabled at st
65          Assert.assertEquals(driver.findElement(strength).getText(), "Not e
66          for (String rule : ALL_RULES) {
67              Assert.assertEquals(ruleState(rule), "pending", "Rule " + rule
68          }
69          Assert.assertEquals(driver.findElement(pwdError).getText(), "");
70          Assert.assertFalse(driver.findElement(successMsg).isDisplayed());
71      }
72
73      // ---------- 2. Positive cases ----------
74      @DataProvider(name = "positive")
75      public Object[][] positive() {
76          return new Object[][]{
77                  // id,     technique,                      password,
78                  {"TC02", "Valid typical password",        "Abcd@1234",
79                  {"TC03", "Boundary: 8 chars (min)",       "Abc@1234",
80                  {"TC04", "Boundary: 16 chars (max)",      "Abcdefgh@123456
81                  {"TC05", "Strong password (12+ chars)",   "Str0ng@Passw0rd
82          };
83      }
84
85      @Test(dataProvider = "positive")
86      public void positiveCases(String id, String technique, String pwd, Str
87          open();
88          enter(pwd);
89
90          SoftAssert soft = new SoftAssert();                  // collect AL
91          for (String rule : ALL_RULES) {
92              soft.assertEquals(ruleState(rule), "pass", id + " rule " + rul
93          }
94          soft.assertEquals(driver.findElement(strength).getText(), expected
95          soft.assertEquals(driver.findElement(pwdError).getText(), "", id +
96          soft.assertTrue(buttonEnabled(), id + " button enabled");
97          soft.assertAll();
98
```

```
        driver.findElement(submitBtn).click();
        Assert.assertEquals(driver.findElement(successMsg).getText(), "Pas
    }

    // ---------- 3. Negative cases: break exactly ONE rule at a time ----
    @DataProvider(name = "negative")
    public Object[][] negative() {
        return new Object[][]{
                // id,     technique,                              passwor
                {"TC06", "Boundary: 7 chars (min - 1)",          "Abc@123"
                {"TC07", "Boundary: 17 chars (max + 1)",         "Abcdefgh
                {"TC08", "Partition: no uppercase letter",       "abcd@123
                {"TC09", "Partition: no lowercase letter",       "ABCD@123
                {"TC10", "Partition: no number",                 "Abcd@efg
                {"TC11", "Partition: no special character",      "Abcd1234
                {"TC12", "Error guessing: space inside",         "Abcd @12
                {"TC13", "Business rule: contains username",     "MyUBED@1
        };
    }

    @Test(dataProvider = "negative")
    public void negativeCases(String id, String technique, String pwd, Str
        open();
        enter(pwd, pwd, user);

        SoftAssert soft = new SoftAssert();
        for (String rule : ALL_RULES) {
            String expected = rule.equals(broken) ? "fail" : "pass";
            soft.assertEquals(ruleState(rule), expected, id + " rule " + r
        }
        soft.assertEquals(driver.findElement(strength).getText(), "Weak",
        soft.assertEquals(driver.findElement(pwdError).getText(), "Passwor
        soft.assertFalse(buttonEnabled(), id + " button must be disabled")
        soft.assertAll();
    }

    // ---------- 4. Error guessing: hostile and strange input ----------
    @Test
    public void tc14_sqlInjectionTextIsRejected() {
        open();
        enter("' OR '1'='1");
        Assert.assertFalse(buttonEnabled());
        Assert.assertFalse(driver.findElement(successMsg).isDisplayed());
    }

    @Test
    public void tc15_scriptTextShowsNoPopup() {
        open();
        enter("<script>alert(1)</script>");
```

```
148         Assert.assertFalse(buttonEnabled());
149         // if the text had run as a script, an alert would be open now
150         try {
151             driver.switchTo().alert();
152             Assert.fail("An unexpected popup appeared");
153         } catch (org.openqa.selenium.NoAlertPresentException expected) {
154             // good: no popup
155         }
156     }
157
158     @Test
159     public void tc16_onlySpacesAreRejected() {
160         open();
161         enter("        ");
162         Assert.assertEquals(ruleState("nospace"), "fail");
163         Assert.assertFalse(buttonEnabled());
164     }
165
166     // ---------- 5. Confirm password ----------
167     @Test
168     public void tc17_mismatchShowsError() {
169         open();
170         enter("Abcd@1234", "Abcd@12345", "");
171         Assert.assertEquals(driver.findElement(confirmError).getText(), "P
172         Assert.assertFalse(buttonEnabled());
173     }
174
175     @Test
176     public void tc18_caseDifferenceIsMismatch() {
177         open();
178         enter("Abcd@1234", "abcd@1234", "");
179         Assert.assertEquals(driver.findElement(confirmError).getText(), "P
180     }
181
182     @Test
183     public void tc19_emptyConfirmKeepsButtonDisabled() {
184         open();
185         enter("Abcd@1234", "", "");
186         Assert.assertFalse(buttonEnabled());
187     }
188
189     // ---------- 6. State transitions ----------
190     @Test
191     public void tc20_validInvalidValidChangesButton() {
192         open();
193         enter("Abcd@1234");
194         Assert.assertTrue(buttonEnabled());
195
196         type(password, "abcd@1234");                        // breaks the
```

```
197         Assert.assertFalse(buttonEnabled());
198
199         type(password, "Abcd@1234");
200         Assert.assertTrue(buttonEnabled());
201     }
202
203     @Test
204     public void tc21_changingUsernameCanBreakValidPassword() {
205         open();
206         enter("Abcd@1234");
207         Assert.assertTrue(buttonEnabled());
208         type(username, "abcd");
209         Assert.assertEquals(ruleState("nouser"), "fail");
210         Assert.assertFalse(buttonEnabled());
211     }
212
213     // ---------- 7. UI behaviour ----------
214     @Test
215     public void tc22_showHideTogglesFieldType() {
216         open();
217         WebElement field = driver.findElement(password);
218         Assert.assertEquals(field.getDomAttribute("type"), "password");
219         driver.findElement(toggleBtn).click();
220         Assert.assertEquals(field.getDomAttribute("type"), "text");
221         Assert.assertEquals(driver.findElement(toggleBtn).getText(), "Hide
222         driver.findElement(toggleBtn).click();
223         Assert.assertEquals(field.getDomAttribute("type"), "password");
224     }
225 }
```

STEP 5: EXPLAINED LINE BY LINE

LINES	WHAT THIS PART DOES
1-9	Package and imports. `SoftAssert` lets one test report all its failures together.
10-12	Extends `BaseTest`, so `driver` and `wait` are ready. Every `@Test` method gets a fresh browser.
13-14	Address of the practice page. Change `PATH/TO` to your folder.
15-17	The seven rule names the page shows. We loop over this array instead of writing seven lines each time.
18-28	Every element is found by its `id` and stored once as a `By`. If an id changes you fix one line.
29-33	`open()` loads the page.

34-41	`type()` finds the box, `clear()` removes old text, `sendKeys()` types the new text. Empty text is skipped.
42-51	`enter()` fills username, password and confirm password. The shorter `enter(pwd)` below it is an overloaded shortcut.
52-55	Reads the `data-state` attribute of a rule line. The page sets it to `pass`, `fail` or `pending`.
56-59	Is the Create password button clickable? Uses `isEnabled()`.
60-72	TC01: before typing anything the button is disabled, strength says "Not entered", every rule is pending and there is no error or success text.
73-84	Data provider with four valid passwords. Two of them are the boundaries 8 and 16 characters. One test method will run four times, once per row.
85-102	Types the password, then checks all rules pass, the strength text, no error and an enabled button. `SoftAssert` reports every failed check together. Then it clicks the button and checks the success message.
103-118	Eight invalid passwords. Each breaks exactly ONE rule: boundary 7 and 17, missing uppercase / lowercase / number / special, a space, and the username rule.
119-134	For every rule it expects `fail` only for the broken rule and `pass` for all others. That proves the page flags the right rule and nothing else. It also checks "Weak", the error text and a disabled button.
135-165	TC14: SQL text. TC15: script text; if it had run, an alert would be open, so `switchTo().alert()` would NOT throw `NoAlertPresentException`. TC16: only spaces.
166-188	Different confirm password, a difference only in letter case, and an empty confirm password.
189-212	The button must follow the data: valid, then invalid, then valid again. Changing the username later can also make a valid password invalid.
213-225	The Show / Hide button changes the field `type` between `password` and `text`, and its own text between Show and Hide.

STEP 6: RUN IT

```
mvn test -Dtest=PasswordTest                  // run this class only
mvn test -Dtest=PasswordTest -Dheadless=true   // run without a browser window
mvn test                                      // run every test
```

▼ Case 2: Full page validation (every kind of input)

Text boxes, date, radio, checkbox, dropdowns, multi-select, slider, textarea, file upload and popups.

STEP 1: THE PRACTICE PAGE

Open the page, click around like a manual tester first, then look at the locators below.

Open demo-form.html

The test already points to this live page (`PAGE_URL`), so it runs from any computer with internet. The same file is in the repository at `docs/demo-pages/demo-form.html` .

IMPORTANT **What you will learn from this case**

- Every input type and how to handle it
- Dependent dropdowns and multi-select
- Alert and confirm popups
- File upload and boundary testing on file size

STEP 2: WHERE EACH ELEMENT IS (LOCATORS)

ELEMENT	HTML ON THE PAGE	BY.ID	CSS SELECTOR	XPATH
Full name box	`<input id="fullName" type="text">`	`fullName`	`#fullName`	`//input[@id='fullName']`
Date of birth	`<input id="dob" type="date">`	`dob`	`input[type='date']`	`//input[@id='dob']`
Male radio button	`<input type="radio" name="gender" id="genderMale" value="Male">`	`genderMale`	`input[name='gender'][value='Male']`	`//input[@name='gender' and @value='Male']`

Country dropdown	`<select id="country">`	`country`	`select#country`	`//select[@id='country']`
State dropdown (starts disabled)	`<select id="state" disabled>`	`state`	`select#state`	`//select[@id='state']`
Skills multi-select	`<select id="skills" multiple>`	`skills`	`select[multiple]`	`//select[@multiple]`
English checkbox	`<input type="checkbox" name="lang" id="langEnglish" value="English">`	`langEnglish`	`input[name='lang']:checked`	`//input[@name='lang' and @value='English']`
Experience slider	`<input type="range" id="experience" min="0" max="20">`	`experience`	`input[type='range']`	`//input[@type='range']`
Address textarea	`<textarea id="address" maxlength="200">`	`address`	`textarea#address`	`//textarea[@id='address']`
Resume file input	`<input type="file" id="resume">`	`resume`	`input[type='file']`	`//input[@type='file']`
Error under a field	`<div class="error" id="err-email">`	`err-email`	`#err-email`	`//div[@id='err-email']`
Submit / Reset buttons	`<button type="submit" id="submitBtn">Submit</button>`	`submitBtn / resetBtn`	`button[type='submit']`	`//button[text()='Reset']`

STEP 3: MANUAL TEST CASES FIRST

Automation follows manual testing. We first decide what to test, and only then write code.

ID	CONTROL	WHAT IS TESTED	TECHNIQUE

TC01-02	Whole form	Start state; submit empty form shows 11 error messages	State check, required fields
TC03	Name, email, phone (text)	Length 1, 2, 40, 41; letters only; email formats; phone length and first digit	Boundary value, equivalence partition
TC04	Date of birth (date)	Exactly 18 years, one day short, one day over, future date	Boundary value
TC05	Gender (radio)	Only one can be selected at a time	State check
TC06-07	Country and State (dependent dropdowns)	State list changes with country; disabled again when country is cleared	Dependency check
TC08	Skills (multi-select)	1 to 3 allowed; 4 and 0 give errors	Boundary value
TC09	Languages (checkboxes)	Tick and untick, count of ticked boxes	State check
TC10	Experience (slider)	Value display at 12 and at maximum 20	Boundary value
TC11-12	Address (textarea)	Counter, 200 character limit, minimum 10 characters	Boundary value
TC13	Resume (file upload)	PDF, PNG, exactly 2 MB, 2 MB + 1 byte, empty file, cv.pdf.exe	Boundary value, error guessing
TC14	Terms (checkbox)	Must be ticked	Required field
TC15	Submit	Full valid form, alert popup, result text	Positive end-to-end
TC16	Reset	Confirm popup: Cancel keeps data, OK clears it	Popup handling

STEP 4: THE CODE

File: `selenium/FormTest.java`

```java
package selenium;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class FormTest extends BaseTest {

    // Live practice page hosted on GitHub Pages (works from any computer
    private static final String PAGE_URL = "https://ubedzz49.github.io/qa-
    private static final int MB = 1024 * 1024;

    // ---------- Helper methods ----------
    private void open() {
        driver.get(PAGE_URL);
    }

    private void type(String id, String text) {
        WebElement field = driver.findElement(By.id(id));
        field.clear();
        field.sendKeys(text);
    }

    private void clickSubmit() {
        driver.findElement(By.id("submitBtn")).click();
    }

    private String error(String field) {
        return driver.findElement(By.id("err-" + field)).getText();
    }

    // Date and range inputs are awkward to type into, so we set them with
    private void setValueWithJs(String id, String value) {
        WebElement el = driver.findElement(By.id(id));
        ((JavascriptExecutor) driver).executeScript(
```

```
                "arguments[0].value = arguments[1];" +
                "arguments[0].dispatchEvent(new Event('input', { bubbles:
    }

    // Create a fake file of a given name and size, return its full path
    private String makeFile(String name, int sizeInBytes) throws IOExcepti
        Path folder = Files.createTempDirectory("upload");
        Path file = folder.resolve(name);
        Files.write(file, new byte[sizeInBytes]);
        return file.toString();
    }

    // ---------- 1. Initial state ----------
    @Test
    public void tc01_initialState() {
        open();
        Assert.assertFalse(driver.findElement(By.id("state")).isEnabled())
        Assert.assertEquals(driver.findElement(By.id("expOut")).getText(),
        Assert.assertEquals(driver.findElement(By.id("addrCount")).getText
        Assert.assertEquals(driver.findElements(By.cssSelector("input[name
        Assert.assertFalse(driver.findElement(By.id("terms")).isSelected()
        Assert.assertFalse(driver.findElement(By.id("result")).isDisplayed
    }

    // ---------- 2. Empty form: every required field shows an error -----
    @Test
    public void tc02_emptyFormShowsAllErrors() {
        open();
        clickSubmit();
        String[][] expected = {
                {"fullName", "Full name is required"},
                {"email", "Email is required"},
                {"phone", "Mobile number is required"},
                {"dob", "Date of birth is required"},
                {"gender", "Select a gender"},
                {"country", "Select a country"},
                {"skills", "Select at least one skill"},
                {"lang", "Select at least one language"},
                {"address", "Address is required"},
                {"resume", "Upload your resume"},
                {"terms", "You must accept the terms"},
        };
        for (String[] row : expected) {
            Assert.assertEquals(error(row[0]), row[1], "Field " + row[0]);
        }
    }

    // ---------- 3. Text inputs: data-driven validation ----------
    @DataProvider(name = "textCases")
```

```
    public Object[][] textCases() {
        String phoneMsg = "Enter a 10 digit number starting with 6, 7, 8 o
        String nameLenMsg = "Name must be 2 to 40 characters";
        return new Object[][]{
                // field,      value,              expected message,  why
                {"fullName", "A",                nameLenMsg, "Boundary: 1
                {"fullName", "Al",               "",         "Boundary: 2
                {"fullName", "A".repeat(40),     "",         "Boundary: 4
                {"fullName", "A".repeat(41),     nameLenMsg, "Boundary: 4
                {"fullName", "John3",            "Name can have only lett
                {"fullName", "Mary-Ann O'Neil",  "",         "Valid speci
                {"email",    "abc",              "Enter a valid email add
                {"email",    "abc@mail",         "Enter a valid email add
                {"email",    "a b@mail.com",     "Enter a valid email add
                {"email",    "user@mail.com",    "",         "Valid email
                {"phone",    "12345",            phoneMsg,   "Invalid: to
                {"phone",    "5876543210",       phoneMsg,   "Invalid: st
                {"phone",    "98765432101",      phoneMsg,   "Boundary: 1
                {"phone",    "abcdefghij",       phoneMsg,   "Invalid: le
                {"phone",    "9876543210",       "",         "Valid mobil
        };
    }

    @Test(dataProvider = "textCases")
    public void tc03_textFieldValidation(String field, String value, Strin
        open();
        type(field, value);
        clickSubmit();
        Assert.assertEquals(error(field), message, why);
    }

    // ---------- 4. Date of birth: boundary around 18 years ----------
    @DataProvider(name = "dobCases")
    public Object[][] dobCases() {
        LocalDate today = LocalDate.now();
        return new Object[][]{
                {"exactly 18 years today (valid)",  today.minusYears(18).t
                {"turns 18 tomorrow (invalid)",     today.minusYears(18).p
                {"18 years and 1 day (valid)",      today.minusYears(18).m
                {"tomorrow (future date)",          today.plusDays(1).toSt
        };
    }

    @Test(dataProvider = "dobCases")
    public void tc04_dateOfBirth(String why, String date, String message)
        open();
        setValueWithJs("dob", date);
        clickSubmit();
        Assert.assertEquals(error("dob"), message, why);
```

```
148     }
149
150     // ---------- 5. Radio buttons ----------
151     @Test
152     public void tc05_onlyOneRadioCanBeSelected() {
153         open();
154         driver.findElement(By.id("genderFemale")).click();
155         Assert.assertTrue(driver.findElement(By.id("genderFemale")).isSele
156         driver.findElement(By.id("genderOther")).click();
157         Assert.assertTrue(driver.findElement(By.id("genderOther")).isSelec
158         Assert.assertFalse(driver.findElement(By.id("genderFemale")).isSel
159         Assert.assertEquals(driver.findElements(By.cssSelector("input[name
160     }
161
162     // ---------- 6. Dropdowns: single and dependent ----------
163     private List<String> optionTexts(Select select) {
164         return select.getOptions().stream().map(WebElement::getText).colle
165     }
166
167     @Test
168     public void tc06_countryChangesStateList() {
169         open();
170         Select country = new Select(driver.findElement(By.id("country")));
171         WebElement stateElement = driver.findElement(By.id("state"));
172
173         country.selectByVisibleText("India");
174         Assert.assertTrue(stateElement.isEnabled());
175         Assert.assertEquals(optionTexts(new Select(stateElement)),
176                 Arrays.asList("-- Select state --", "Karnataka", "Maharash
177
178         country.selectByValue("USA");
179         Assert.assertEquals(optionTexts(new Select(stateElement)),
180                 Arrays.asList("-- Select state --", "California", "Texas")
181
182         country.selectByIndex(0);                              // back to "-
183         Assert.assertFalse(stateElement.isEnabled());
184     }
185
186     @Test
187     public void tc07_stateIsRequiredAfterChoosingCountry() {
188         open();
189         new Select(driver.findElement(By.id("country"))).selectByValue("UK
190         clickSubmit();
191         Assert.assertEquals(error("state"), "Select a state");
192     }
193
194     // ---------- 7. Multi-select ----------
195     @Test
196     public void tc08_multiSelectAllowsOneToThreeSkills() {
```

```
        open();
        Select skills = new Select(driver.findElement(By.id("skills")));
        Assert.assertTrue(skills.isMultiple());

        skills.selectByValue("Selenium");
        skills.selectByValue("Git");
        clickSubmit();
        Assert.assertEquals(error("skills"), "");

        skills.selectByValue("Playwright");
        skills.selectByValue("SQL");                          // now 4 sele
        clickSubmit();
        Assert.assertEquals(error("skills"), "Select at most 3 skills");

        skills.deselectAll();
        clickSubmit();
        Assert.assertEquals(error("skills"), "Select at least one skill");
    }

    // ---------- 8. Checkboxes ----------
    @Test
    public void tc09_languageCheckboxes() {
        open();
        driver.findElement(By.id("langEnglish")).click();
        driver.findElement(By.id("langTelugu")).click();
        Assert.assertEquals(driver.findElements(By.cssSelector("input[name
        driver.findElement(By.id("langEnglish")).click();      // click ag
        Assert.assertEquals(driver.findElements(By.cssSelector("input[name
    }

    // ---------- 9. Slider ----------
    @Test
    public void tc10_sliderShowsItsValue() {
        open();
        setValueWithJs("experience", "12");
        Assert.assertEquals(driver.findElement(By.id("expOut")).getText(),
        setValueWithJs("experience", "20");                    // max
        Assert.assertEquals(driver.findElement(By.id("expOut")).getText(),
    }

    // ---------- 10. Textarea ----------
    @Test
    public void tc11_textareaCounterAndLimit() {
        open();
        WebElement box = driver.findElement(By.id("address"));
        box.sendKeys("Hello");
        Assert.assertEquals(driver.findElement(By.id("addrCount")).getText

        box.clear();
```

```
246         box.sendKeys("x".repeat(205));                          // we type
247         Assert.assertEquals(box.getAttribute("value").length(), 200);
248         Assert.assertEquals(driver.findElement(By.id("addrCount")).getText
249     }
250 
251     @Test
252     public void tc12_shortAddressIsRejected() {
253         open();
254         type("address", "Short");
255         clickSubmit();
256         Assert.assertEquals(error("address"), "Address must be at least 10
257     }
258 
259     // ---------- 11. File upload ----------
260     @DataProvider(name = "fileCases")
261     public Object[][] fileCases() {
262         return new Object[][]{
263                 // why,                       file name,      size,
264                 {"valid PDF",               "resume.pdf",  1000,        ""
265                 {"wrong type (png)",        "photo.png",   1000,        "O
266                 {"exactly 2 MB (max)",      "big.pdf",     2 * MB,      ""
267                 {"2 MB + 1 byte (max + 1)", "bigger.pdf",  2 * MB + 1,  "F
268                 {"empty file",              "empty.pdf",   0,           "F
269                 {"double extension",        "cv.pdf.exe",  1000,        "O
270         };
271     }
272 
273     @Test(dataProvider = "fileCases")
274     public void tc13_resumeUpload(String why, String name, int size, Strin
275         open();
276         // For a file input we do NOT click. We send the file path as text
277         driver.findElement(By.id("resume")).sendKeys(makeFile(name, size))
278         clickSubmit();
279         Assert.assertEquals(error("resume"), message, why);
280     }
281 
282     // ---------- 12. Terms checkbox ----------
283     @Test
284     public void tc14_termsMustBeAccepted() {
285         open();
286         clickSubmit();
287         Assert.assertEquals(error("terms"), "You must accept the terms");
288         driver.findElement(By.id("terms")).click();
289         clickSubmit();
290         Assert.assertEquals(error("terms"), "");
291     }
292 
293     // ---------- 13. Happy path: whole form, plus the alert popup -------
294     @Test
```

```
    public void tc15_validFormSubmitsAndShowsResult() throws IOException {
        open();
        type("fullName", "Ravi Kumar");
        type("email", "ravi@example.com");
        type("phone", "9876543210");
        setValueWithJs("dob", "1998-05-20");
        driver.findElement(By.id("genderMale")).click();

        new Select(driver.findElement(By.id("country"))).selectByVisibleTe
        new Select(driver.findElement(By.id("state"))).selectByVisibleText

        Select skills = new Select(driver.findElement(By.id("skills")));
        skills.selectByValue("Selenium");
        skills.selectByValue("Playwright");

        driver.findElement(By.id("langEnglish")).click();
        driver.findElement(By.id("langHindi")).click();
        setValueWithJs("experience", "2");
        type("address", "12 MG Road, Bengaluru");
        driver.findElement(By.id("resume")).sendKeys(makeFile("ravi.pdf",
        driver.findElement(By.id("terms")).click();

        clickSubmit();

        // The page shows an alert popup. We must handle it before doing a
        Alert alert = wait.until(ExpectedConditions.alertIsPresent());
        Assert.assertEquals(alert.getText(), "Application submitted succes
        alert.accept();

        String json = driver.findElement(By.id("result")).getText();
        Assert.assertTrue(json.contains("\"fullName\": \"Ravi Kumar\""));
        Assert.assertTrue(json.contains("\"phone\": \"9876543210\""));
        Assert.assertTrue(json.contains("\"state\": \"Karnataka\""));
        Assert.assertTrue(json.contains("\"experience\": 2"));
        Assert.assertTrue(json.contains("\"resume\": \"ravi.pdf\""));
    }

    // ---------- 14. Reset button and the confirm popup ----------
    @Test
    public void tc16_resetAsksForConfirmation() {
        open();
        type("fullName", "Ravi Kumar");
        new Select(driver.findElement(By.id("country"))).selectByValue("US

        driver.findElement(By.id("resetBtn")).click();
        Alert confirm = wait.until(ExpectedConditions.alertIsPresent());
        Assert.assertEquals(confirm.getText(), "Clear the whole form?");
        confirm.dismiss();                                    // press Ca
        Assert.assertEquals(driver.findElement(By.id("fullName")).getAttri
```

```
344
345         driver.findElement(By.id("resetBtn")).click();
346         wait.until(ExpectedConditions.alertIsPresent()).accept();    // pre
347         Assert.assertEquals(driver.findElement(By.id("fullName")).getAttri
348         Assert.assertFalse(driver.findElement(By.id("state")).isEnabled())
349     }
350 }
```

STEP 5: EXPLAINED LINE BY LINE

LINES	WHAT THIS PART DOES
1-20	Imports. Note `Select`, `Alert`, `JavascriptExecutor`, `LocalDate` and file classes. Each is used for one type of input.
21-26	Extends `BaseTest`. `PAGE_URL` is the page address. `MB` is the number of bytes in 1 MB, used for the file size boundaries.
27-46	Small helpers: `open()`, `type()`, `clickSubmit()` and `error(field)`, which reads the red message under a field (`err-<field>`).
47-54	A date box and a slider are hard to type into. This helper sets the value with JavaScript, then fires an `input` event so the page reacts as if a user did it.
55-61	Creates a real temporary file with a given name and size. A file input needs a real path on disk.
62-73	TC01: the State dropdown is disabled, counters show 0, no radio selected, terms not ticked, result hidden.
74-96	TC02: click Submit on an empty form. A 2D array holds each field and its expected message, and a loop checks all 11.
97-121	Data provider with 15 rows: boundaries on name length (1, 2, 40, 41), invalid characters, email formats, phone length and first digit. An empty expected message means "no error expected".
122-129	One method for all 15 rows: type the value into the field whose id is `field`, click Submit, compare the error text.
130-149	Dates are calculated from today with `LocalDate`, so the test never goes out of date. Boundary: exactly 18 years, one day too young, one day older, and a future date.
150-161	Click a radio, check `isSelected()`. Picking another one must unselect the first. A CSS selector with `:checked` counts selected radios.

162-193	`Select` wraps a `<select>`. This is a dependent dropdown: changing the country changes the list of states. `getOptions()` reads the list; `selectByIndex(0)` returns to the first option and the State box becomes disabled again.
194-215	`isMultiple()` confirms the list allows many choices. Select 2 (valid), then 4 (max + 1, error), then `deselectAll()` (0, error).
216-226	Clicking a checkbox toggles it. Count of ticked boxes uses the CSS `:checked`.
227-236	The slider value is set with JavaScript and the shown number is checked, including the maximum 20.
237-258	Types "Hello" and checks the counter. Then types 205 characters: the browser keeps only 200 (`maxlength`). Another test checks the 10 character minimum.
259-281	Six cases with name and size. Boundary: exactly 2 MB passes, 2 MB + 1 byte fails. Also a wrong type, an empty file, and a double extension (`cv.pdf.exe`).
282-292	Without ticking the terms an error shows. After ticking it, the error clears.
293-331	Fills every control with valid data. After Submit the page shows an alert; `alertIsPresent()` waits for it, `getText()` reads it, `accept()` presses OK. Then the result text is checked.
332-350	Reset shows a confirm popup. `dismiss()` (Cancel) keeps the data. `accept()` (OK) clears the form and disables the State box.

STEP 6: RUN IT

```
mvn test -Dtest=FormTest                  // run this class only
mvn test -Dtest=FormTest -Dheadless=true   // run without a browser window
mvn test                                  // run every test
```

▼ Case 3: Complete flow (login to order confirmation)

One user journey: login, search, cart, coupon, checkout, payment and logout.

STEP 1: THE PRACTICE PAGE

Open the page, click around like a manual tester first, then look at the locators below.

Open demo-shop.html

The test already points to this live page (`PAGE_URL`), so it runs from any computer with internet. The same file is in the repository at `docs/demo-pages/demo-shop.html` .

IMPORTANT **What you will learn from this case**

- An end-to-end test with many steps
- Checking calculations (subtotal, discount, total)
- Negative cases inside a flow
- Finding elements inside a card or table row

STEP 2: WHERE EACH ELEMENT IS (LOCATORS)

ELEMENT	HTML ON THE PAGE	BY.ID	CSS SELECTOR	XPATH
Login email box	`<input id="email" type="email">`	`email`	`#email`	`//input[@id='email']`
Login button	`<button type="button" id="loginBtn">Login </button>`	`loginBtn`	`#loginBtn`	`//button[text()='Login']`
Search box	`<input id="search" placeholder="Search products">`	`search`	`input[placeholder='Search products']`	`//input[@placeholder='Search products']`
A product card	`<div class="product" id="product-2">`	`product-2`	`.product`	`//div[@id='product-2']`
Add button inside a card	`<button id="add-3">Add to cart</button>`	`add-3`	`#product-3 button`	`//div[@id='product-3']//button`
Cart link	`<button id="cartLink">Cart (<span id="cartCount">0</span>)</button>`	`cartLink`	`#cartLink`	`//button[contains(text(),'Cart')]`
Quantity of an item	`<span id="qty-1">2</span>`	`qty-1`	`#cart-row-1 span`	`//tr[@id='cart-row-1']//span`

Remove button	`<button id="rm-3">Remove</button>`	`rm-3`	`#rm-3`	`//tr[@id='cart-row-3']//button[text()='Remove']`
Coupon box	`<input id="couponInput" aria-label="Coupon code">`	`couponInput`	`#couponInput`	`//input[@id='couponInput']`
Total amount	`<strong id="total">`	`total`	`#total`	`//strong[@id='total']`
UPI radio button	`<input type="radio" name="pay" id="payUpi" value="UPI">`	`payUpi`	`input[name='pay'][value='UPI']`	`//input[@name='pay' and @value='UPI']`
Order id	`<strong id="orderId">`	`orderId`	`#orderId`	`//strong[@id='orderId']`

STEP 3: MANUAL TEST CASES FIRST

Automation follows manual testing. We first decide what to test, and only then write code.

STEP	ACTION	EXPECTED RESULT	IDEA
Login (4 cases)	Empty fields, wrong password, unknown email, password in wrong case	Correct error message, shop page stays hidden	Negative testing
1	Login with qa@test.com and Test@1234	Welcome text, 3 products shown	Positive
2	Search "pen", "zzz"; filter by category	1 product, "No products found", counts correct	Search and filter
3	Add Notebook twice and Backpack once	Cart count is 3	State change
4	Open the cart	Notebook qty 2, line total 500, subtotal 1700	Calculation check
5	Click + then -	Quantity 3 then back to 2	State change

6	Coupon ABC, then SAVE10	Invalid message; then discount 170, total 1530	Negative then positive
7	Remove Backpack	Subtotal 500, discount 50, total 450	Recalculation
8	Checkout with empty fields, PIN 123, no payment	Three different error messages	Validation
9	Pay with UPI	Order placed, id like ORD-12345, total 450, cart emptied	End result
10	Logout	Back to login page	Flow end
Extra	Open cart with no items	Checkout button disabled	Rule check

STEP 4: THE CODE

File: `selenium/ShopFlowTest.java`

```java
package selenium;

import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class ShopFlowTest extends BaseTest {

    // Live practice page hosted on GitHub Pages (works from any computer
    private static final String PAGE_URL = "https://ubedzz49.github.io/qa-
    private static final String EMAIL = "qa@test.com";
    private static final String PASSWORD = "Test@1234";

    // ---------- Helper methods ----------
    private void type(String id, String text) {
        WebElement field = driver.findElement(By.id(id));
        field.clear();
        field.sendKeys(text);
    }

    private void click(String id) {
        driver.findElement(By.id(id)).click();
    }

    private String text(String id) {
        return driver.findElement(By.id(id)).getText();
    }

    private void login(String email, String password) {
        type("email", email);
        type("pass", password);
        click("loginBtn");
    }

    // "₹1530" → 1530
    private int money(String id) {
        return Integer.parseInt(text(id).replace("₹", ""));
    }

    private int productCount() {
        return driver.findElements(By.cssSelector(".product")).size();
    }

    // Empty a text box by pressing Backspace (this fires the key events t
```

```
    private void emptyBox(String id) {
        WebElement box = driver.findElement(By.id(id));
        int length = box.getAttribute("value").length();
        for (int i = 0; i < length; i++) {
            box.sendKeys(Keys.BACK_SPACE);
        }
    }

    // ---------- Login: negative cases ----------
    @DataProvider(name = "badLogins")
    public Object[][] badLogins() {
        return new Object[][]{
                {"both empty",             "",                "",
                {"wrong password",         EMAIL,             "wrong",
                {"unknown email",          "nobody@test.com",  PASSWORD,
                {"password in wrong case", EMAIL,             "test@1234"
        };
    }

    @Test(dataProvider = "badLogins")
    public void loginFails(String why, String email, String password, Stri
        driver.get(PAGE_URL);
        login(email, password);
        Assert.assertEquals(text("loginError"), message, why);
        Assert.assertFalse(driver.findElement(By.id("shopView")).isDisplay
    }

    // ---------- Complete end-to-end flow ----------
    @Test
    public void endToEndPurchase() {
        driver.get(PAGE_URL);

        // 1. Login with valid details
        login(EMAIL, PASSWORD);
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("sh
        Assert.assertEquals(text("welcome"), "Welcome, qa@test.com");
        Assert.assertEquals(productCount(), 3);

        // 2. Search and filter products
        type("search", "pen");
        Assert.assertEquals(productCount(), 1);
        Assert.assertEquals(driver.findElement(By.cssSelector("#product-2

        emptyBox("search");
        type("search", "zzz");
        Assert.assertTrue(driver.findElement(By.id("noResults")).isDisplay

        emptyBox("search");
        Select category = new Select(driver.findElement(By.id("category"))
```

```
        category.selectByVisibleText("Bags");
        Assert.assertEquals(productCount(), 1);
        category.selectByValue("All");
        Assert.assertEquals(productCount(), 3);

        // 3. Add products to the cart
        click("add-1");                                  // Notebook
        click("add-1");                                  // Notebook again
        driver.findElement(By.xpath("//div[@id='product-3']//button")).cli
        Assert.assertEquals(text("cartCount"), "3");

        // 4. Check the cart and the maths
        click("cartLink");
        Assert.assertEquals(text("qty-1"), "2");
        Assert.assertEquals(driver.findElement(By.cssSelector("#cart-row-1
        Assert.assertEquals(money("subtotal"), 500 + 1200);
        Assert.assertEquals(money("discount"), 0);
        Assert.assertEquals(money("total"), 1700);

        // 5. Change quantity
        click("inc-1");
        Assert.assertEquals(text("qty-1"), "3");
        click("dec-1");
        Assert.assertEquals(text("qty-1"), "2");

        // 6. Coupon: invalid first, then valid
        type("couponInput", "ABC");
        click("applyCoupon");
        Assert.assertEquals(text("couponMsg"), "Invalid coupon code");
        Assert.assertEquals(money("discount"), 0);

        type("couponInput", "SAVE10");
        click("applyCoupon");
        Assert.assertEquals(text("couponMsg"), "Coupon SAVE10 applied: 10%
        Assert.assertEquals(money("discount"), 170);
        Assert.assertEquals(money("total"), 1530);

        // 7. Remove an item and the total updates
        click("rm-3");                                   // remove Backpac
        Assert.assertEquals(driver.findElements(By.id("cart-row-3")).size(
        Assert.assertEquals(money("subtotal"), 500);
        Assert.assertEquals(money("discount"), 50);
        Assert.assertEquals(money("total"), 450);

        // 8. Checkout: validation first
        click("checkoutBtn");
        click("placeOrder");
        Assert.assertEquals(text("checkoutError"), "Name and address are r

```

```
148          type("shipName", "Ravi Kumar");
149          type("shipAddress", "12 MG Road, Bengaluru");
150          type("pin", "123");
151          click("placeOrder");
152          Assert.assertEquals(text("checkoutError"), "Enter a valid 6 digit
153
154          type("pin", "560001");
155          click("placeOrder");
156          Assert.assertEquals(text("checkoutError"), "Select a payment metho
157
158          // 9. Pay and confirm the order
159          click("payUpi");
160          click("placeOrder");
161          wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("do
162          Assert.assertEquals(text("orderStatus"), "Order placed");
163          Assert.assertTrue(text("orderId").matches("ORD-\\d{5}"), "Order id
164          Assert.assertEquals(text("orderTotal"), "₹450");
165          Assert.assertEquals(text("orderPay"), "UPI");
166          Assert.assertEquals(text("cartCount"), "0");     // cart is emptie
167
168          // 10. Logout returns to the login page
169          click("logoutBtn");
170          Assert.assertTrue(driver.findElement(By.id("loginView")).isDisplay
171          Assert.assertFalse(driver.findElement(By.id("topNav")).isDisplayed
172      }
173
174      // ---------- Empty cart rule ----------
175      @Test
176      public void checkoutIsDisabledForEmptyCart() {
177          driver.get(PAGE_URL);
178          login(EMAIL, PASSWORD);
179          click("cartLink");
180          Assert.assertTrue(driver.findElement(By.id("emptyCart")).isDisplay
181          Assert.assertFalse(driver.findElement(By.id("checkoutBtn")).isEnab
182      }
183  }
```

STEP 5: EXPLAINED LINE BY LINE

LINES	WHAT THIS PART DOES
1-11	Imports. `Keys` is used to press Backspace.
12-18	Extends `BaseTest`. The page address and the valid login are constants.
19-40	Short helpers so the test reads like a story: `type`, `click`, `text` (visible text by id), and `login`.

41-49	`money("total")` turns the text "₹1530" into the number 1530 so we can compare numbers.
50-57	Empties a box by pressing Backspace. This fires the keyboard events the search box listens to.
58-68	Four wrong logins: both empty, wrong password, unknown email, and the password in the wrong letter case.
69-76	Each row logs in with bad data, then checks the exact error text and that the shop page stays hidden.
77-81	One long test is one full user journey. The numbered comments are the steps.
82-87	Log in, wait until the shop page is visible (explicit wait), check the welcome text and 3 products.
88-103	Search "pen" shows 1 product; a CSS selector with a space (`#product-2 .name`) finds the name inside the card. "zzz" shows "No products found". The `Select` class filters by category.
104-109	Add the Notebook twice. The Backpack button is found with XPath: the button inside the div whose id is `product-3` .
110-117	Open the cart and check quantity 2, the line total, and that subtotal 1700 = 500 + 1200.
118-123	Plus makes quantity 3, minus brings it back to 2.
124-135	A wrong coupon gives an error and no discount. SAVE10 gives 10% off: discount 170, total 1530.
136-142	Remove the Backpack. The row must be gone (`findElements` size 0) and the totals recalculate: 500, 50, 450.
143-157	Three validation messages in order: empty fields, wrong PIN 123, no payment method.
158-167	Choose UPI and place the order. Check the status, the order id with a regular expression (`ORD-` and 5 digits), the amount, and that the cart is empty.
168-173	Logout returns to the login page and hides the top menu.
174-183	A separate small test: with an empty cart the Checkout button must be disabled.

STEP 6: RUN IT

```
mvn test -Dtest=ShopFlowTest                 // run this class only
mvn test -Dtest=ShopFlowTest -Dheadless=true   // run without a browser window
mvn test                                     // run every test
```

---

13. Quick interview answers

- IMPORTANT **What is Selenium?**
  An open-source library to automate web browsers. It supports many languages (Java, Python, C#) and browsers (Chrome, Firefox, Edge, Safari). It does not test desktop or mobile apps (mobile uses Appium).
- IMPORTANT **Why `WebDriver driver = new ChromeDriver()` and not `ChromeDriver driver = ...`?**
  WebDriver is an interface. Writing the interface on the left lets you switch to another browser by changing one word on the right.
- IMPORTANT **findElement vs findElements?**
  One element and an exception if none, versus a list that is empty if none.
- IMPORTANT **close() vs quit()?**
  close() closes the current window. quit() closes all windows and ends the session.
- IMPORTANT **Implicit vs explicit wait?**
  Implicit: global, for all elements. Explicit: one condition for one element, with `WebDriverWait`. Prefer explicit, and never mix the two.
- IMPORTANT **Which locator do you prefer?**
  id first, then CSS, then relative XPath, because they are fast and stable.
- IMPORTANT **XPath vs CSS?**
  XPath can search by text and move to parent elements. CSS is simpler and usually faster.
- IMPORTANT **Absolute vs relative XPath?**
  Absolute starts from the root (`/html/...`) and breaks easily. Relative starts with `//` and is safer.
- IMPORTANT **How do you handle a dropdown, an alert, a frame, a new window?**
  `Select` class; `switchTo().alert()` ; `switchTo().frame()` ; `getWindowHandles()` + `switchTo().window()` .

- IMPORTANT **How do you handle StaleElementReferenceException?**
  The page changed and the saved element is old. Find the element again.
- IMPORTANT **What is Page Object Model?**
  A design pattern: one class per page that holds its locators and actions. It reduces repeated code and makes maintenance easy.
- IMPORTANT **How does TestNG help?**
  Runs tests, gives annotations (`@Test` , `@BeforeMethod`), assertions, data providers, grouping, parallel runs and reports.
- GOOD TO KNOW **What is a data provider?**
  A method that returns rows of data. The test method runs once per row.
- GOOD TO KNOW **How do you upload a file?**
  `sendKeys` with the file path on the `<input type="file">` . Do not click it.
- GOOD TO KNOW **How do you run tests without a visible browser?**
  Headless mode with `ChromeOptions.addArguments("--headless=new")` .
- GOOD TO KNOW **What is Selenium Grid?**
  Runs tests in parallel on many browsers and machines.
- GOOD TO KNOW **What are Selenium limitations?**
  Web only, no built-in reports, needs explicit waits, and cannot handle captcha or image checks.
- GOOD TO KNOW **Selenium vs Playwright?**
  Playwright has built-in auto-waiting, is often faster, and has built-in tracing. Selenium has the bigger community and more language support. Learn both ideas.